INCA RAIL

PRUEBA TÉCNICA

# Senior Full Stack Developer

Laravel 13 · PHP 8.5 · Vue 3

LA RUTA DE LA PRUEBA




Versión 4.0 · Octubre 2026 · Plazo: 2 días (48 horas)

# 1. Antes de empezar

Gracias por tomarte el tiempo de hacer esta prueba para el puesto de Senior Full Stack Developer en Inca Rail.

Vas a construir una API en Laravel y una aplicación en Vue que, juntas, muestran datos de fútbol de un servicio externo (football-data.org). El ejercicio es pequeño a propósito: lo que nos interesa es ver cómo piensas, cómo ordenas tu código y cómo explicas lo que hiciste.

No esperamos que hagas todo. Una parte es obligatoria y otra es opcional. Preferimos lo obligatorio bien resuelto antes que muchos opcionales a medias.

## 1.1 La prueba de un vistazo

| Tema | Obligatorio | Opcional |
|---|---|---|
| Backend (Laravel) | API con Laravel 13 y PHP 8.5; Passport; usuarios, roles y permisos; los 5 endpoints; datos guardados con Eloquent; manejo del límite de 10 peticiones por minuto; errores bien respondidos. | DDD, arquitectura hexagonal o Clean Architecture (o ninguna, argumentando), CQRS, EDA; pruebas y TDD; integración continua; colas, jobs o cron. |
| Frontend (Vue) | SPA con Vue 3, responsive, con login, ligas, equipos y jugadores, y gestión de usuarios, roles y permisos. | Vuetify, Pinia, Composition API y TypeScript, componentes reutilizables, CASL. |
| Documentación | Diagrama(s) de arquitectura en la carpeta docs/ y README.md. | Modelo de datos y diagramas UML. |
| Infraestructura | Ninguna. | Docker (compose.yml), Vagrant u otra forma de levantar el proyecto; Linux. |
| Entrega | Un solo correo con el enlace al repositorio. | — |

## 1.2 Cómo leer los requisitos

Cada requisito tiene un ID (por ejemplo, B3 o F2) con el que luego calificamos, y una etiqueta de color:

| Etiqueta | Qué significa |
|---|---|
| **OBLIGATORIO** | Hay que hacerlo. Si falta o queda a medias, baja tu puntaje. Los mínimos de la sección 1.5 son todavía más importantes. |
| **OPCIONAL** hasta +n pts | No es necesario. Si lo haces bien, suma hasta los puntos que indica (máximo +20 en total). No lo hagas a costa de un obligatorio. Un opcional mal hecho no resta. |

## 1.3 Reglas del juego

- **Plazo:** 2 días (48 horas) desde que recibes la prueba. Hay una sola entrega, por correo (sección 4).
- **Trabajo individual.** Puedes consultar documentación y usar herramientas de apoyo, incluida IA. Lo que entregues debe ser algo que entiendas y puedas explicar.

- **Tu propio token.** Crea una cuenta gratuita en football-data.org; el token queda asociado a tu cuenta. No lo subas al repositorio.
- **Dudas.** Escríbele al evaluador. Si algo no queda claro y no alcanzas respuesta, decide, anota tu supuesto en el README y sigue.
- **Si no llegas a todo,** entrega lo que tengas y cuéntanos en el README qué faltó y cómo lo harías.

### 1.4 Tecnologías que usarás

La prueba se resuelve con este stack. Si quieres sumar otras herramientas, adelante, siempre que las versiones base sean estas:

| Tecnología | Condición |
|---|---|
| PHP 8.5 | OBLIGATORIO |
| Laravel 13, con Laravel Passport (versión compatible) | OBLIGATORIO |
| Vue.js 3 con Vue Router | OBLIGATORIO |
| Base de datos a tu elección (SQL Server, MySQL, PostgreSQL u otra), siempre con Eloquent | OBLIGATORIO |
| Vuetify, Pinia, TypeScript, CASL | OPCIONAL |
| Docker, Vagrant, Linux | OPCIONAL |

### 1.5 Mínimos para que revisemos tu entrega

Antes de leer tu código revisamos estos seis puntos. Si alguno falla, el puntaje base máximo baja a 60 de 100.

| N.º | Mínimo |
|---|---|
| 1 | Se puede ejecutar siguiendo solo el README, sin pasos ocultos. |
| 2 | Usas PHP 8.5, Laravel 13 y Vue 3, y guardas los datos con Eloquent (migraciones y modelos) en la base de datos que elijas. |
| 3 | No hay secretos en el repositorio (ni el token de football-data). Hay un .env.example. |
| 4 | La autenticación usa Passport, las contraseñas tienen hash y las rutas están protegidas. |
| 5 | Existe la carpeta docs/ con al menos un diagrama de arquitectura, y el README lo referencia. |
| 6 | Entregaste en un solo correo, dentro del plazo, con enlaces que funcionan. |

## 2. El caso

Imagina que el equipo necesita una plataforma interna para consultar ligas, equipos y jugadores de fútbol. La información viene de un servicio externo, football-data.org (https://www.football-data.org), pero queremos conservar una copia propia de lo que se consulta. Además, no todo el mundo puede hacer lo mismo: cada persona tiene un rol con permisos distintos.

Tu trabajo: un API REST en Laravel que consulta football-data.org, guarda lo que necesita y controla quién puede hacer qué; una SPA en Vue para iniciar sesión, ver los datos y administrar usuarios, roles y permisos; y la documentación para que cualquiera entienda tu arquitectura y pueda ejecutar el proyecto.

**Un detalle importante: el límite de 10 peticiones por minuto**

El token gratuito de football-data solo permite 10 peticiones por minuto. Una solución que funciona con tres clics pero se cae al cuarto no nos sirve.

Queremos ver cómo lo resuelves y, sobre todo, que nos expliques tu decisión en el README.

# 3. Qué debes construir

## 3.1 Backend: API REST en Laravel

| ID | Requisito | Tipo |
|---|---|---|
| B1 | **Autenticación con Laravel Passport**<br>Login con correo y contraseña que entregue un token de acceso, y logout que lo invalide. Todas las rutas, salvo el login, van protegidas. Las contraseñas se guardan con hash. | OBLIGATORIO |
| B2 | **Usuarios, roles y permisos (ACL)**<br>• Un módulo para gestionar usuarios, roles y permisos: crear y editar usuarios, asignarles roles y desactivarlos.<br>• Lo que puede hacer cada rol se decide en el servidor, no solo en la interfaz.<br>• El API entrega al frontend los permisos del usuario en un formato que se pueda usar con CASL (o con la librería que elijas).<br>• Incluye seeders con, al menos, un rol de administrador y uno de solo consulta. | OBLIGATORIO |
| B3 | **Los cinco endpoints**<br>• GET /competitions: detalle de todas las ligas.<br>• GET /competitions/{competition_id}: detalle de una liga. Al consultarla, sus equipos y jugadores se guardan en tu base de datos.<br>• GET /team: todos los equipos guardados hasta el momento.<br>• GET /team/{team_id}: detalle de un equipo.<br>• GET /players: todos los jugadores guardados hasta el momento.<br>• Respeta las rutas tal como están escritas. | OBLIGATORIO |
| B4 | **Datos locales con Eloquent**<br>Usa la base de datos que prefieras (SQL Server, MySQL, PostgreSQL u otra). Lo que importa es que trabajes con Eloquent: migraciones, modelos y relaciones, sin SQL escrito a mano. De cada jugador debes guardar nombre, posición y número de camiseta. Si consultas la misma liga dos veces, no debe duplicar registros. | OBLIGATORIO |
| B5 | **Consumo de football-data.org y límite de peticiones**<br>• El token gratuito permite 10 peticiones por minuto. Piensa cómo evitar que ese límite rompa tu API (datos guardados, caché, esperas entre llamadas, una cola…).<br>• Las consultas a football-data deben ser independientes de las consultas a tu propia API: lo que ya guardaste se tiene que poder responder aunque la API externa no esté disponible o no tenga cupo en ese momento.<br>• El token va en el archivo .env, nunca en el código.<br>• Cuéntanos en el README qué decidiste y por qué. | OBLIGATORIO |

| ID | Requisito | Tipo |
|---|---|---|
| B6 | **Respuestas correctas ante errores**<br>Respuestas en JSON, siempre con el mismo formato y con el código HTTP que corresponde: equipo o jugador que no existe (404), datos inválidos (422), sin sesión o sin permiso (401 o 403), límite de peticiones excedido (429) y pérdida de conexión con football-data (502, 503 o equivalente). | OBLIGATORIO |
| B7 | **Organización del código**<br>Controladores delgados, lógica en servicios o clases dedicadas, validación con Form Requests y respuestas con Resources. Lo que habla con football-data va aislado en una sola clase. | OBLIGATORIO |

## 3.2 Frontend: SPA en Vue

Aquí tienes libertad para agregar lo que creas relevante para mostrar la información. Lo único que no puede faltar es lo que está en esta tabla.

| ID | Requisito | Tipo |
|---|---|---|
| F1 | **SPA con Vue 3**<br>Aplicación de una sola página con Vue.js 3 y Vue Router. Debe ser responsive: que se vea bien en el celular y en el escritorio. | OBLIGATORIO |
| F2 | **Inicio de sesión**<br>Pantalla de login contra tu API. Si la sesión vence o falta un permiso, la persona ve un mensaje claro, no una pantalla rota. | OBLIGATORIO |
| F3 | **Ligas, equipos y jugadores**<br>Pantallas para ver las competiciones, importar una liga (consultar su detalle) y ver el listado y el detalle de los equipos y jugadores guardados. | OBLIGATORIO |
| F4 | **Usuarios, roles y permisos**<br>Pantallas para gestionar usuarios, roles y permisos. Lo que cada persona ve y puede hacer depende de sus permisos. | OBLIGATORIO |
| F5 | **Experiencia de uso**<br>Estados de carga, listas vacías y errores bien resueltos, con mensajes entendibles cuando football-data falla o se llega al límite de peticiones. Aquí tienes libertad para sumar lo que creas útil (búsqueda, filtros, paginación…). | OBLIGATORIO |

## 3.3 Documentación y diagramas

| ID | Requisito | Tipo |
|---|---|---|
| D1 | **Diagrama(s) de arquitectura en la carpeta docs/**<br>• Crea una carpeta docs/ en la raíz del repositorio con el diagrama de arquitectura de tu solución. Puedes hacer más de uno.<br>• Debe mostrar, como mínimo: la SPA, tu API Laravel, la base de datos, football-data.org, la autenticación con Passport y cómo manejas el límite de 10 peticiones por minuto.<br>• Guarda la imagen (PNG, SVG o PDF) y, si la tienes, el archivo editable (draw.io, Mermaid, etc.). | OBLIGATORIO |

| ID | Requisito | Tipo |
|---|---|---|
| D2 | **README.md**<br>• Cómo ejecutar el proyecto paso a paso: requisitos, instalación, variables de entorno, migraciones y usuarios de prueba.<br>• Cómo abordaste la solución y las decisiones que tomaste, incluido el manejo del límite de peticiones.<br>• Cada diagrama de docs/ incrustado o enlazado, con dos o tres líneas que lo expliquen.<br>• Los supuestos que hiciste y lo que quedó pendiente. | **OBLIGATORIO** |

> **Cómo debería verse tu carpeta docs/**
>
> Por ejemplo: `docs/arquitectura.png` (o .svg) y `docs/arquitectura.drawio` (.mmd si usas Mermaid o .puml si usas plantuml). Si haces más de un diagrama, ponles nombres que se entiendan solos: `docs/flujo-importacion.png`, `docs/modelo-datos.png`.
>
> En el README, cada diagrama aparece con un título y una explicación corta de qué muestra y por qué lo planteaste así.

## 3.4 Lo opcional

Nada de esta sección es necesario. Elige lo que te permita mostrar mejor cómo trabajas. Cada opcional indica los puntos máximos que puede sumar; el total de bonus tiene un tope de +20.

### Arquitectura y patrones

*Aquí no hay una única respuesta correcta. Puedes aplicar arquitectura hexagonal, Clean Architecture, una combinación de ambas o ninguna. Lo que cuenta es que elijas con criterio y expliques por qué: decidir «ninguna» porque el proyecto es pequeño es una decisión válida si la argumentas. No premiamos el patrón por sí mismo ni el sobrediseño.*

| ID | Requisito | Tipo |
|---|---|---|
| O1 | **DDD (Domain-Driven Design)**<br>Diseña la solución con enfoque de dominio: contextos, entidades, servicios de dominio. | **OPCIONAL**<br>hasta +1.5 pts |
| O2 | **Arquitectura hexagonal o Clean Architecture**<br>Si aplicas alguna de las dos, muestra cómo aíslas el dominio de Laravel y de football-data (puertos y adaptadores, casos de uso, capas) y por qué esa y no la otra. | **OPCIONAL**<br>hasta +2 pts |
| O3 | **CQRS**<br>Separa comandos y consultas donde aporte valor. | **OPCIONAL**<br>hasta +1 pts |
| O4 | **Arquitectura orientada a eventos (EDA)**<br>Maneja los eventos del sistema con este enfoque (por ejemplo, un evento cuando se importa una liga). | **OPCIONAL**<br>hasta +1 pts |
| O5 | **Decisión de arquitectura argumentada**<br>En el README, compara las opciones que consideraste (hexagonal, Clean Architecture, capas simples de Laravel u otra) y explica por qué elegiste una de ellas o ninguna. Vale lo mismo elegir «ninguna» con buenos argumentos. | **OPCIONAL**<br>hasta +1.5 pts |
| O6 | **SOLID y Clean Code**<br>Que se note de forma consistente en todo el código. | **OPCIONAL**<br>hasta +0.5 pts |

### Pruebas

*Pruebas automáticas que se puedan ejecutar con un comando documentado.*

| ID | Requisito | Tipo |
|---|---|---|
| O7 | **Pruebas unitarias**<br>Cubren la lógica importante (por ejemplo, los servicios y el manejo del límite de peticiones). | **OPCIONAL**<br>hasta +1.5 pts |
| O8 | **Pruebas de integración**<br>Prueban la API con base de datos, incluidos los permisos por rol. | **OPCIONAL**<br>hasta +1.5 pts |
| O9 | **Pruebas de aceptación**<br>Validan flujos completos desde el punto de vista del usuario. | **OPCIONAL**<br>hasta +1 pts |
| O10 | **TDD**<br>Que el historial de commits muestre pruebas escritas antes del código en algunas funcionalidades. | **OPCIONAL**<br>hasta +1 pts |

### Integración continua

| ID | Requisito | Tipo |
|---|---|---|
| O11 | **Pipelines de CI**<br>Los que creas necesarios (análisis de estilo, pruebas, build del frontend). Elige el proveedor que uses para el repositorio: GitLab CI, GitHub Actions u otro. | **OPCIONAL**<br>hasta +3 pts |

### Frontend moderno

| ID | Requisito | Tipo |
|---|---|---|
| O12 | **Vuetify**<br>Como framework de diseño. | **OPCIONAL**<br>hasta +0.5 pts |
| O13 | **Pinia**<br>Para la gestión del estado. | **OPCIONAL**<br>hasta +1 pts |
| O14 | **Composition API y TypeScript**<br>Implementación moderna y tipada. | **OPCIONAL**<br>hasta +1.5 pts |
| O15 | **Componentes reutilizables**<br>Componentes pensados para usarse en varias pantallas. | **OPCIONAL**<br>hasta +0.5 pts |
| O16 | **ACL con CASL**<br>Permisos aplicados en rutas y elementos de la interfaz a partir de lo que entrega tu API. | **OPCIONAL**<br>hasta +1 pts |

### Distribución del software (Docker, Linux u otra)

*Que otra persona del equipo pueda levantar el proyecto con las mismas herramientas que tú. La infraestructura no es obligatoria.*

| ID | Requisito | Tipo |
|---|---|---|
| O17 | **Docker (compose.yml), Vagrant u otra herramienta**<br>Con Docker, un compose.yml que levante todo. Si además usas Linux con Nginx y PHP-FPM, mejor. | **OPCIONAL**<br>hasta +3 pts |

### Colas y tareas programadas

| ID | Requisito | Tipo |
|---|---|---|
| O18 | **Colas con Jobs, Listeners y Events, o Cron Jobs**<br>Por ejemplo, para importar ligas en segundo plano respetando el límite de peticiones, o para refrescar datos cada cierto tiempo. | **OPCIONAL**<br>hasta +2 pts |

### Documentación extra

| ID | Requisito | Tipo |
|---|---|---|
| O19 | **Modelo de datos y diagramas UML**<br>Modelo lógico y físico de datos, diagrama de secuencia de la importación de una liga, u otros que ayuden a entender tu solución. | **OPCIONAL**<br>hasta +1 pts |

## 3.5 Recomendaciones

- Tómate un rato para **analizar y diseñar** antes de programar. Un buen diagrama al inicio te ahorra tiempo después.
- Si piensas hacer integración continua, elige primero dónde vivirá tu repositorio (GitHub o GitLab), porque de eso depende cómo la configuras.
- Una cola con Jobs, Listeners y Events, o un Cron Job, puede ayudarte a respetar el límite de peticiones. Es una idea, no una exigencia.
- Si te ayuda a explicarte, suma el modelo de datos o un diagrama UML. Lo valoramos, pero no es obligatorio.
- Si al final te alcanza tiempo y deseas exponer tu desarrollo en un ambiente productivo, genial, suma.

# 4. Cómo entregar

Envía un solo correo al evaluador cuando termines. Ese correo es tu entrega; después no revisaremos cambios.

| Parte del correo | Qué debe tener |
|---|---|
| Asunto | Prueba técnica Senior Full Stack - Tu nombre |
| Enlaces | El enlace a tu repositorio en GitHub o GitLab. Si separas backend y frontend, un enlace por cada uno. Verifica que se puedan abrir (si son privados, dale acceso al evaluador). |
| Resumen breve | Dos o tres líneas: qué opcionales hiciste, qué supuestos tomaste y qué quedó pendiente. |

**Estructura sugerida del repositorio**

| Ruta | Qué contiene |
|---|---|
| README.md | Cómo ejecutar, cómo abordaste la solución, decisiones, supuestos y diagramas referenciados. |
| docs/ | Diagrama(s) de arquitectura (imagen y archivo editable) y, si quieres, modelo de datos y UML. |
| backend/ y frontend/ | El API Laravel y la SPA Vue (o dos repositorios separados, si lo prefieres). |
| .env.example | Variables necesarias, sin valores secretos. |
| compose.yml | Solo si eliges Docker (opcional). |

# 5. Cómo evaluamos

Cada criterio se califica con una rúbrica de 0 a 4. El nivel 3, «sólido», es lo que esperamos de un perfil senior. Estos son los pesos:

| N.º | Criterio | Qué miramos | Puntos |
|---|---|---|---|
| 1 | Backend (API Laravel) | Passport; usuarios, roles y permisos; endpoints; datos con Eloquent; consumo de football-data y límite de peticiones; respuestas ante errores; organización del código. | 40 |
| 2 | Frontend (SPA Vue) | Base de la SPA y sesión; funcionalidad; experiencia de uso y responsive; estructura y reutilización. | 25 |
| 3 | Arquitectura y documentación | Diagrama(s) en docs/; README claro y reproducible; coherencia entre lo que dibujas y lo que programas. | 20 |
| 4 | Calidad del código | Legibilidad, seguridad básica y rendimiento básico. | 10 |
| 5 | Repositorio y entrega | Commits con sentido, repositorio ordenado, entrega en un solo correo y en plazo. | 5 |

| N.º | Criterio | Qué miramos | Puntos |
|---|---|---|---|
| | Total base | Los opcionales suman hasta +20 puntos adicionales. | 100 |

## 6. Revisa esto antes de enviar

- ☐ Probé el README desde cero (en una carpeta limpia) y el proyecto se ejecuta.
- ☐ Hay usuarios de prueba documentados, con distintos roles.
- ☐ Las cinco rutas funcionan y respondo bien cuando un equipo o jugador no existe.
- ☐ Probé qué pasa cuando se alcanza el límite de 10 peticiones por minuto, y lo expliqué en el README.
- ☐ El token de football-data no está en el código ni en el historial; existe .env.example.
- ☐ La carpeta docs/ tiene mi diagrama de arquitectura y el README lo muestra o lo enlaza.
- ☐ Mis commits cuentan una historia y no subí node_modules ni vendor.
- ☐ Mi correo lleva los enlaces, los opcionales que hice y mis supuestos.

**¡Mucho éxito! Si algo del enunciado no te queda claro, escríbenos antes de asumir.**